

=====

[기말 프로젝트 보고서]

과목명: 보안네트워크프로그래밍

팀명: 타조

팀원: 이동훈, 정유진, 추민호

=====



[목차]

1. 프로젝트 개요
 - 1.1 프로그램 목표 및 기획 의도
 - 1.2 프로그램의 필살기
 - 1.3 조원들의 역할
2. 프로그램 실행 방법 및 실행 화면
 - 2.1 프로그램 실행 방법
 - 2.2 프로그램 실행 화면
3. 프로그램 설계 및 코드 설명
 - 3.1 프로토콜 소개 및 설명
 - 3.2 SSL 보안 채널 구현
 - 3.3 로그인 및 HOTP 구현
 - 3.3.1 회원가입 구현
 - 3.3.2 로그인 구현
 - 3.4 퀴즈 진행 구현
 - 3.4.1 로비 구현
 - 3.4.2 퀴즈 구현
4. 결론 및 느낀 점

1. 프로젝트 개요

1.1 프로그램 목표 및 기획 의도

본 프로젝트는 '어떻게 하면 재미있게 공부할 수 있을까?' 하는 의문에서 시작되었다. 친구들과 함께 공부할 때, 공부가 잘 되었는지 확인하기 위해 '아무거나 문제 내봐, 맞춰 볼게.' 하는 경우가 있다. 또한 Kahoot, Quizlet 등 퀴즈 프로그램들이 존재하는데, 이에 착안하여 학습 보조 퀴즈 프로그램을 목표로 하게 되었다.

1.2 프로그램의 필살기

프로그램의 필살기는 다음과 같다.

첫째, '친구들과 함께' 공부하기 위한 환경일 것. 이를 위해서는 동시에 같은 문제를 풀어야 하고, 친구들의 상태를 확인할 수 있어야 한다. 프로그램에서는 방장이 퀴즈를 선택했을 때 모두가 같은 문제를 동시에 풀도록 설계되어 있으며, 친구들의 실시간 점수를 통해 자신이 얼마나 잘하고 있는지 확인할 수 있다. 또한 퀴즈의 점수는 같이 문제를 풀고 있는 사람이 많을수록 많이 오르도록 설계되어 있어, 함께 공부할 것을 유도할 수 있다.

둘째, '재미있게' 공부하기 위한 환경일 것. 본 프로그램에서는 재미를 최고기록에서 찾았다. 각 퀴즈별 최고기록을 경신하기 위해 경쟁하고, 프로그램 특성상 동시에 학습하는 사람이 많을수록 점수가 더 많이 오름을 이용하여 재미를 유도한다. 또한 방장이 퀴즈를 업로드할 수 있기 때문에 공부와 함께 재미를 위한 문제도 석을 수 있다.

셋째, '공정한' 공부 환경일 것. 네트워크상으로 패킷을 주고받기 때문에, 실제 시험에서의 부정행위처럼 오가는 패킷을 훔쳐보고 답을 확인할 위험이 있다. 이를 차단하기 위해 SSL 보안 채널을 도입하였다. 또한 점수가 공정하게 오르도록 race condition을 방지하였다.

1.3 조원들의 역할

'타조'는 이동훈, 정유진, 추민호 학생으로 구성되어 있다.

이동훈 학생은 퀴즈 진행 프로토콜을 구현하고, 정유진 학생의 코드 병합 역할을 도왔다.

정유진 학생은 SSL 보안 채널을 구현하고 퀴즈 진행 내에서 mutex lock을 사용하였으며, 서버 측 코드의 구조를 짜고 이동훈 학생과 추민호 학생의 코드를 병합하였다.

추민호 학생은 회원가입, 로그인 및 HOTP를 구현하였다.

2. 프로그램 실행 방법 및 실행 화면

2.1 프로그램 실행 방법

Server와 Client 측에는 아래와 같은 파일이 필요하다. Makefile에 컴파일 명령이 적혀 있으므로, 양 측 모두 make를 입력하여 컴파일을 진행할 수 있다.

이후 Server 측은 ./server, Client 측은 ./client를 입력해 프로그램을 실행시킬 수 있다.

▼ server

▼ certificates
ca.crt
ca.key
ca.srl
server.crt
server.csr
server.key
account.c
account.h
client_handler.c
client_handler.h
die_with_message.c
die_with_message.h
highscores.db
lobby.c
lobby.h
main.c
Makefile
quiz.c
quiz.h
security_programming.q
users.db
util.c
util.h

```
CC = gcc
CFLAGS = -Wall -I.
LIBS = -lssl -lcrypto -lpthread

# 서버 실행에 필요한 소스 파일들
SRCS = main.c client_handler.c account.c quiz.c util.c lobby.c die_with_message.c
OBJS = $(SRCS:.c=.o)

TARGET = server

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) -o $@ $^ $(LIBS)

# .c 파일을 .o 파일로 컴파일하는 규칙
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f *.o $(TARGET)
```

▼ client

> certificates
client_users.db
client.c
die_with_message.c
die_with_message.h
Makefile

```
CC = gcc
CFLAGS = -Wall -I.
LIBS = -lssl -lcrypto

SRCS = client.c die_with_message.c
OBJS = $(SRCS:.c=.o)

TARGET = client

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) -o $@ $^ $(LIBS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f *.o $(TARGET)
```

2.2 프로그램 실행 화면

```
$ ./client
서버 IP 입력 : 127.0.0.1
[client] SSL handshake complete

Let's Quiz

Welcome to Security Programming Quiz!

-----
명령 : 1)REGISTER 2)LOGIN : 2
ID: dhsama51
PW: dhsama51
>> 로그인 성공 ! (Counter: 6)

=== [QUIZ LOBBY] ===
[Lobby] 현재 접속자 수 : 1명

[ADMIN] 당신은 방장입니다 !
[ADMIN] 퀴즈 주제를 선택하세요 :
1) security_programming.q(20문제)
U) 새 퀴즈 파일 업로드
번호 또는 'U' 입력 > |
```

```
user@LAPTOP-DONGHOON ~/project/client
$ ./client
서버 IP 입력 : 127.0.0.1
[client] SSL handshake complete

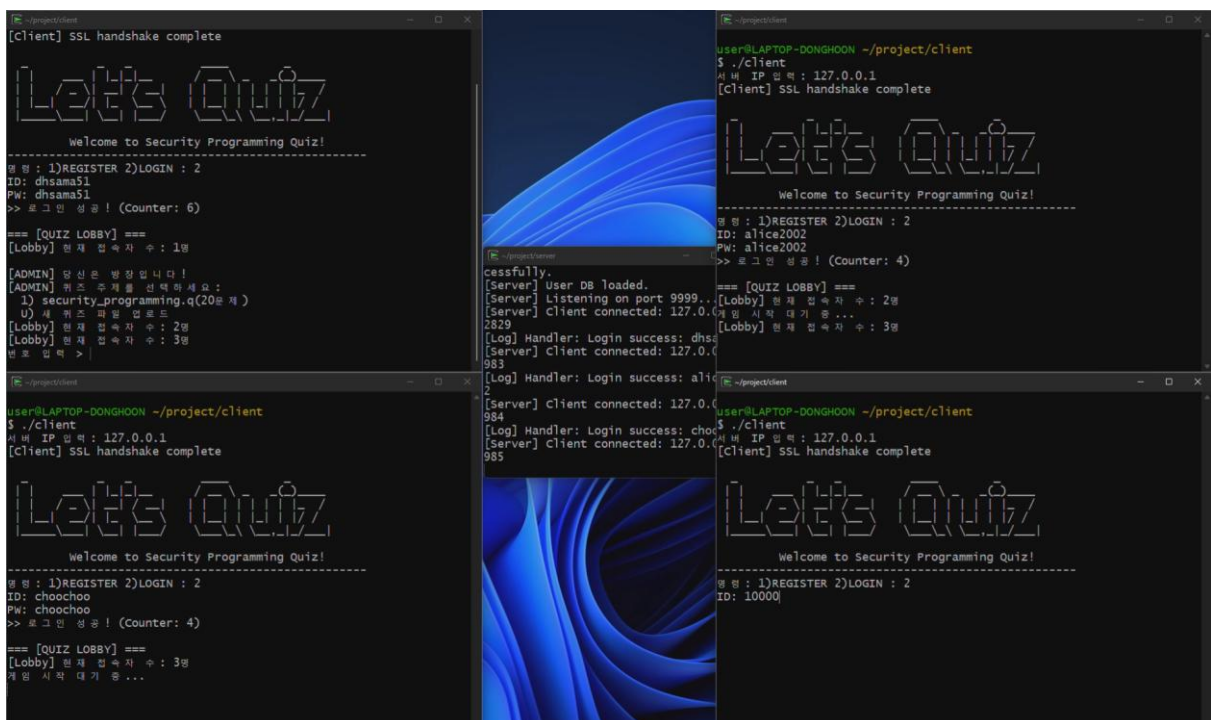
Let's Quiz

Welcome to Security Programming Quiz!

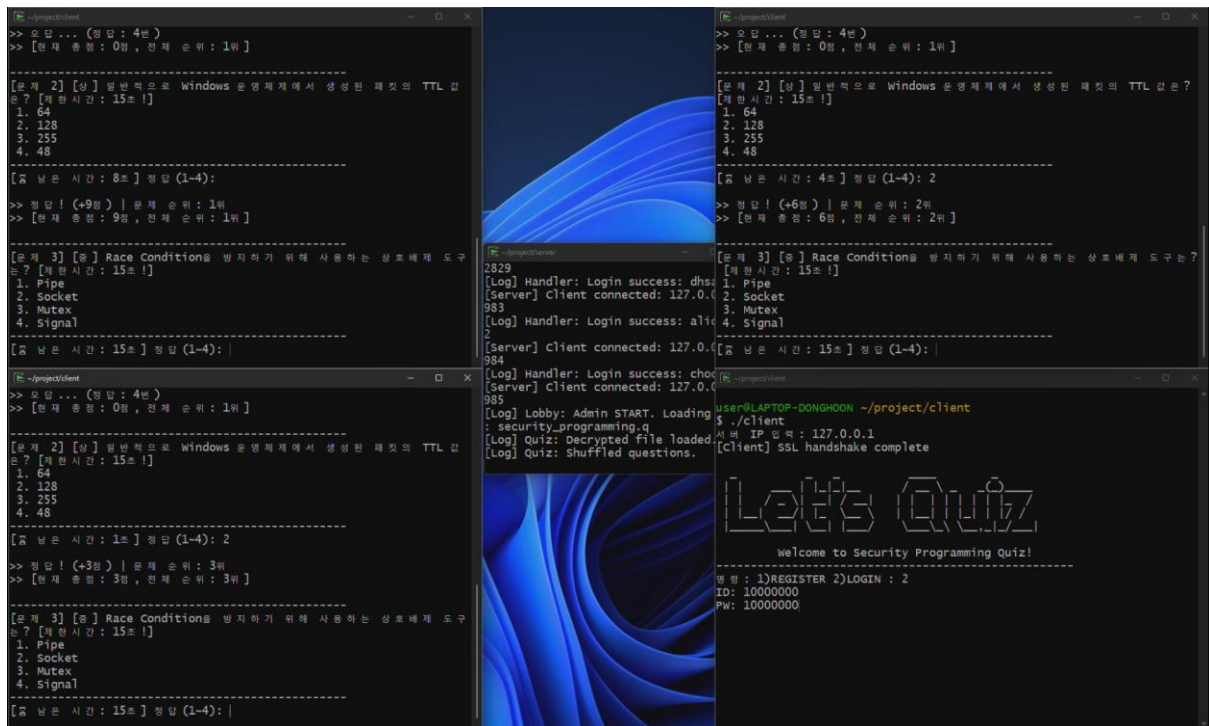
-----
명령 : 1)REGISTER 2)LOGIN : 2
ID: alice2002
PW: alice2002
>> 로그인 성공 ! (Counter: 4)

=== [QUIZ LOBBY] ===
[Lobby] 현재 접속자 수 : 2명
게임 시작 대기 중 ...
```

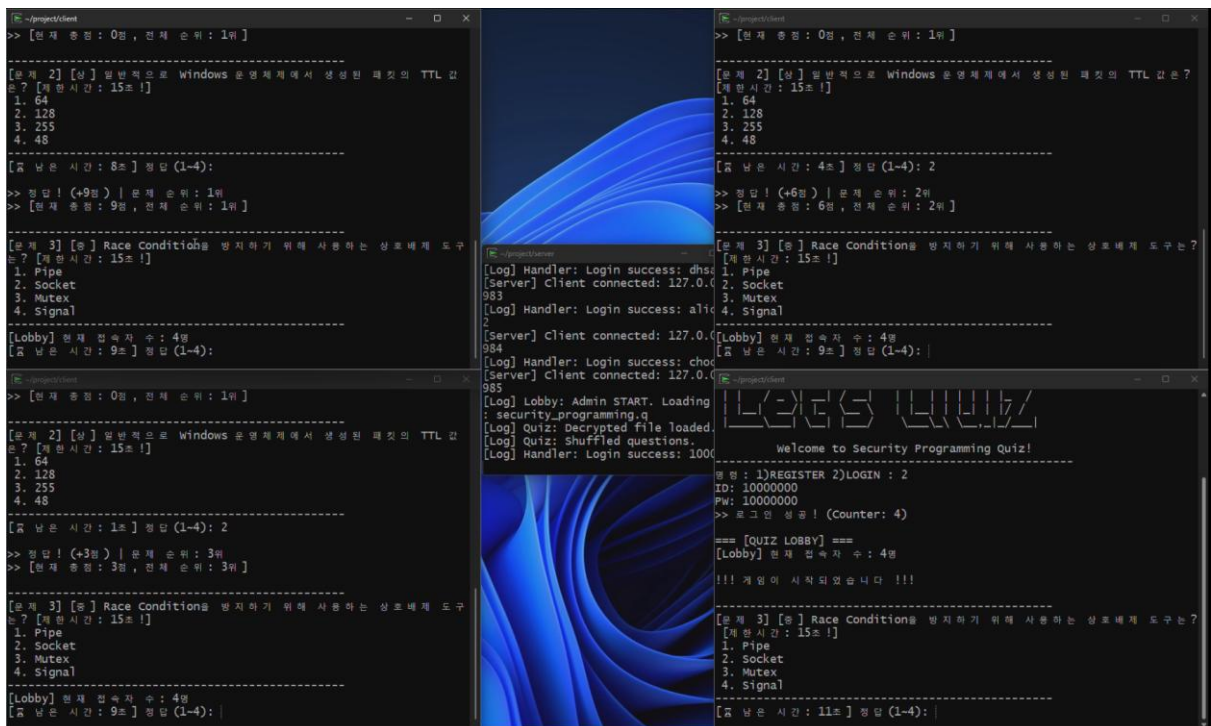
이후 Server 측은 ./server, Client 측은 ./client를 입력하여 프로그램을 실행시킬 수 있다. 위 사진과 같이, 첫 번째로 입장한 Client는 방장 권한을 획득하여 퀴즈 주제를 선택함과 동시에 퀴즈를 시작할 수 있다. 이후에 입장한 Client는 방장 Client가 퀴즈를 시작하기를 대기한다.



우측 하단 Client만 접속하지 못한 상태이고, 방장 Client는 3명 이상이 접속하면 퀴즈를 시작할 수 있다.

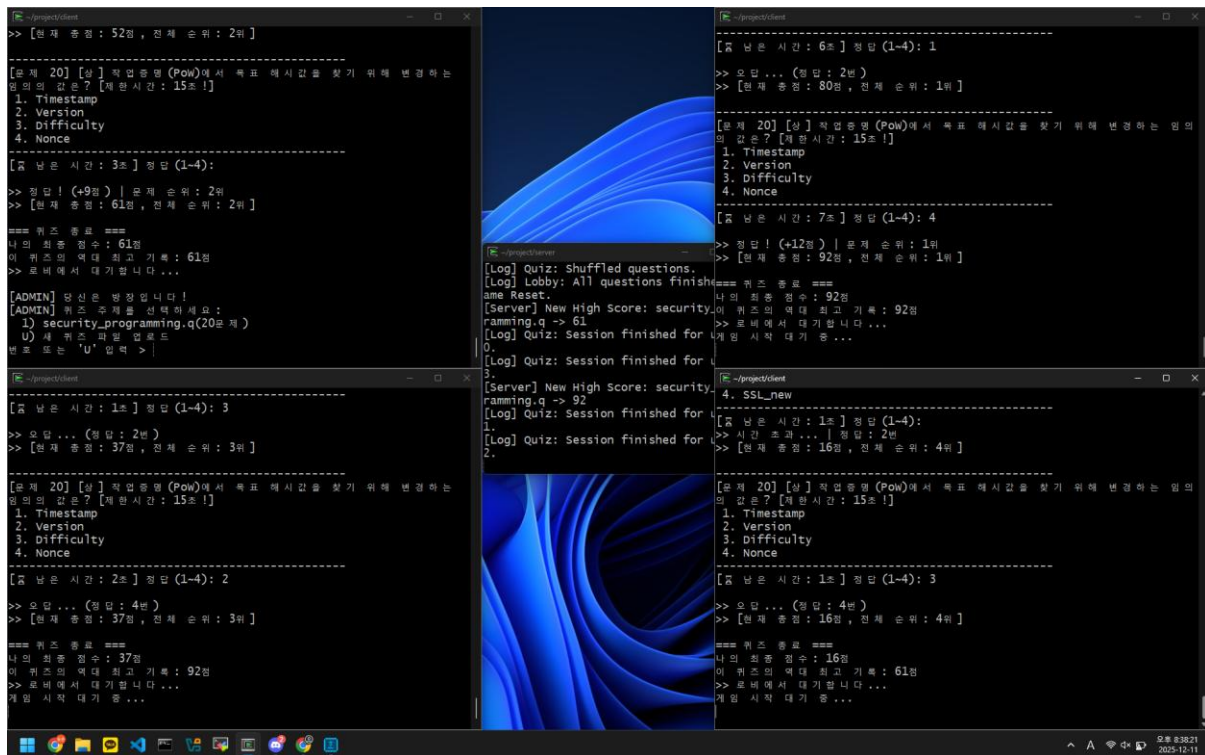


퀴즈가 진행되면, 다음과 같이 문제, 선지가 출력되고 제한 시간 15초 내에 정답을 입력하여 전송해야 한다. 이후 정답/오답 여부와 함께 획득 점수, 현재 점수, 현재 등수를 확인할 수 있다.



늦게 접속한 친구가 있더라도 문제 없이 진행되며, 해당 Client는 당시에 진행되던 문

제부터 같이 풀 수 있다.



퀴즈가 마무리되고 나면, 최종 등수, 나의 최종 점수, 이 퀴즈의 역대 최고 기록을 확인할 수 있다. 이를 통해 경쟁심을 자극할 수 있다.

3. 프로그램 설계 및 코드 설명

3.1 프로토콜 소개 및 설명

SSL 보안 채널 부분의 프로토콜은 강의자료의 흐름과 동일하다. SSL 채널을 정리할 때, 동적 할당했던 메모리를 모두 해제하여 메모리 누수를 방지하였다.

회원가입, 로그인 및 HOTP 부분의 프로토콜은 다음과 같다.

ID와 패스워드가 과도하게 만들어지는 것을 방지하기 위해, 정해진 학번을 통해 1개의 계정씩만 생성할 수 있도록 설계하였다. 또한 로그인 절차에서 중복 로그인 등도 방지하여 부정행위를 방지하였다.

퀴즈 진행 부분의 프로토콜은 다음과 같다.

로비에서 대기하다가, 방장이 퀴즈를 업로드하거나 시작하고, 퀴즈가 끝나면 다시 로비로 돌아오는 구조이다. 중간에 참여하는 Client들도 당시에 풀던 문제부터 점수를 받을 수 있다.

3.2 SSL 보안 채널 구현

```
<Server> main.c
SSL_CTX* init_server_ctx() {
    SSL_load_error_strings();
    OpenSSL_add_ssl_algorithms();

    SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
    if (!ctx) {
        DieWithOpenSSLError("Failed to create SSL_CTX");
    }

    if (SSL_CTX_use_certificate_file(ctx, "certificates/server.crt",
SSL_FILETYPE_PEM) <= 0 ||
        SSL_CTX_use_PrivateKey_file(ctx, "certificates/server.key",
SSL_FILETYPE_PEM) <= 0)
    {
        DieWithOpenSSLError("Failed to load certificates");
    }

    return ctx;
}

struct sockaddr_in caddr;
socklen_t len = sizeof(caddr);

int client_sock = accept(serv_sock, (struct sockaddr*)&caddr, &len);
if (client_sock < 0) {
    PrintSystemError("accept() failed");
    continue;
}

printf("[Server] Client connected: %s:%d\n",
        inet_ntoa(caddr.sin_addr), ntohs(caddr.sin_port));

SSL *ssl = SSL_new(ctx);
SSL_set_fd(ssl, client_sock);
```

```

    if (SSL_accept(ssl) <= 0) {
        PrintOpenSSLError("SSL_accept() failed");
        SSL_free(ssl);
        close(client_sock);
        continue;
    }

    client_param_t *param = malloc(sizeof(client_param_t));
    if (!param) {
        PrintSystemError("malloc() failed");
        SSL_free(ssl);
        close(client_sock);
        continue;
    }
    param->ssl = ssl;
    param->sock = client_sock;

    pthread_t tid;
    if (pthread_create(&tid, NULL, client_thread, param) != 0) {
        PrintSystemError("pthread_create() failed");
        SSL_free(ssl);
        close(client_sock);
        free(param);
    } else {
        pthread_detach(tid);
    }

    ...
    SSL_CTX_free(ctx);
    close(serv_sock);
    EVP_cleanup();
    return 0;

```

위 코드를 통하여 서버 인증서를 업로드하고 SSL 보안 채널을 초기화한다. 중간에 발생할 수 있는 예외들도 충실하게 처리하였다.

<Client>

```

SSL_library_init(); SSL_load_error_strings(); OpenSSL_add_all_algorithms();
printf("서버 IP 입력: "); char ip[64]; scanf("%63s", ip);

int sock = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in addr = {0}; addr.sin_family = AF_INET; addr.sin_port =
htons(PORT);
inet_pton(AF_INET, ip, &addr.sin_addr);

if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
    DieWithSystemMessage("connect() failed");
}

```

```

SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
if (SSL_CTX_load_verify_locations(ctx, "certificates/ca.crt", NULL)!=1) {
    DieWithOpenSSLError("Failed to load CA certificate");
}

SSL *ssl = SSL_new(ctx); SSL_set_fd(ssl, sock);
if (SSL_connect(ssl) <= 0) {
    DieWithOpenSSLError("SSL_connect() failed");
}

printf("[Client] SSL handshake complete\n");

print_welcome();

```

Client에서는 SSL 보안 채널을 구축한 뒤 print_welcome을 통하여 웰컴 메시지를 출력한다.

3.3 회원가입, 로그인 및 HOTP 구현

3.3.1 회원가입 구현

```

<Client>
int handle_client_register(SSL *ssl, char *out_id, char *out_pw) {
    //REGISTER 메시지
    char sid[64]; printf("학번 입력(SID): "); scanf("%63s", sid);
    char msg[256]; sprintf(msg, "REGISTER %s\n", sid); SSL_write(ssl, msg,
strlen(msg));

    //REGISTER_OK 메시지 수신
    char buf[1024]; int n = recv_line(ssl, buf, sizeof(buf));
    if (n <= 0 || strcmp(buf, "REGISTER_OK", 11) != 0) { printf("가입 실패: %s",
buf); return 0; }

    //CREATE 메시지
    printf("사용할 ID: "); scanf("%63s", out_id); printf("사용할 PW: ");
scanf("%63s", out_pw);
    sprintf(msg, "CREATE %s %s\n", out_id, out_pw); SSL_write(ssl, msg,
strlen(msg));

    //CREATE_OK 메시지 수신
    n = recv_line(ssl, buf, sizeof(buf));
    char cmd[16], r[64];

    if (sscanf(buf, "%15s %63s", cmd, r)==2 && strcmp(cmd, "CREATE_OK")==0) {

```

```

        save_local(out_id, hex_to_uint64(r), 0); printf("회원가입 완료!
(ID: %s)\n", out_id); return 1;
    } printf("가입 실패: %s", buf); return 0;
}

```

Client는 REGISTER, CREATE을 전송하고 REGISTER_OK, CREATE_OK를 수신한다.

<Server> account.c

```

char sid[16];
//REGISTER 메시지 수신
if (sscanf(line, "REGISTER %15s", sid) != 1) {
    PrintUserMessage("Register", "Invalid format");
    ssl_send(ssl, "REGISTER_FAIL Invalid\n");
    return -1;
}
...
//REGISTER_OK 메시지
ssl_send(ssl, "REGISTER_OK\n");

char buf[256];
//CREATE 메시지 수신
if (ssl_readline(ssl, buf, sizeof(buf)) <= 0) return -1;
char cmd[16], id[MAX_ID_LEN], pw[MAX_PW_LEN];
if (sscanf(buf, "%15s %63s %63s", cmd, id, pw) != 3 ||
strcmp(cmd, "CREATE") != 0) {
    PrintUserMessage("Register", "CREATE format error");
    ssl_send(ssl, "CREATE_FAIL InvalidFormat\n");
    return -1;
}
...

unsigned char pw_hash[32];
SHA256((unsigned char*)pw, strlen(pw), pw_hash);

for (int i = 0; i < MAX_USERS; i++) {
    if (!g_users[i].used) {
        User *u = &g_users[i];
        u->used = 1; strcpy(u->sid, sid); strcpy(u->id, id);
        memcpy(u->pw_hash, pw_hash, 32);
        RAND_bytes((unsigned char*)&u->R, sizeof(uint64_t));
        u->counter = 0;
        save_users();
        char R_hex[17]; uint64_to_hex(u->R, R_hex);
        //CREATE_OK 메시지
        char out[64]; sprintf(out, "CREATE_OK %s\n", R_hex);
        ssl_send(ssl, out);
        return 0;
    }
}
}

```

```
PrintUserMessage("Register", "No user slots available"); // [수정]
ssl_send(ssl, "CREATE_FAIL NoSlot\n");
return -1;
```

서버는 REGISTER, CREATE를 수신하고, REGISTER_OK, CREATE_OK를 전송한다.

3.3.2 로그인 구현

```
<Client>
int perform_login(SSL *ssl, const char *id, const char *pw) {
    uint64_t R, counter;
    // 1. 로컬 DB 에서 R, Counter 로드
    if (!load_local(id, &R, &counter)) { ... return 0; }

    // 2. HOTP 값(세션 해시) 계산
    unsigned char pw_hash[32];
    SHA256((unsigned char*)pw, strlen(pw), pw_hash); // PW 해시

    uint64_t V = R + counter; // 가변 값 V 생성
    unsigned char Vbuf[8];
    for (int i=0; i<8; i++) Vbuf[7-i]=(V>>(i*8))&0xff;

    unsigned char calc[40];
    memcpy(calc, pw_hash, 32);
    memcpy(calc+32, Vbuf, 8);
    unsigned char sess[32];
    SHA256(calc, 40, sess); // 세션 해시 = SHA256(PW 해시 + V)

    // 3. LOGIN 메시지 전송
    char phex[65], shex[65];
    bytes_to_hex(pw_hash, 32, phex);
    bytes_to_hex(sess, 32, shex);
    char msg[512];
    sprintf(msg, "LOGIN %s %s %s\n", id, phex, shex);
    SSL_write(ssl, msg, strlen(msg)); //LOGIN {id} {pw_hash} {sess_hash}

    // 4. 결과 확인
    char buf[256]; int n = recv_line(ssl, buf, sizeof(buf));
    if (n > 0 && strncmp(buf, "LOGIN_OK", 8) == 0) { // LOGIN_OK 수신
        printf(">> 로그인 성공! (Counter: %llu)\n", (unsigned long long)counter);
        save_local(id, R, counter+1); // 카운터 증가 후 저장
        return 1;
    } ...
}
```

Client는 LOGIN을 전송한다.

```
<Server> account.c
```

```

static int handle_login(SSL *ssl, const char *line, char *out_id) {
    char id[64], pw_hash_hex[65], sess_hash_hex[65];
    // [수신] LOGIN {id} {pw_hash} {sess_hash} 파싱
    if (sscanf(line, "LOGIN %63s %64s %64s", id, pw_hash_hex, sess_hash_hex) !=
3) ...

    // ... 사용자 검색 (u) ...

    // 1. 비밀번호 해시 검증
    unsigned char pw_hash_client[32], sess_hash_client[32];
    hex_to_bytes(pw_hash_hex, pw_hash_client, 32);
    hex_to_bytes(sess_hash_hex, sess_hash_client, 32);

    if (memcmp(pw_hash_client, u->pw_hash, 32)!=0) {
        ssl_send(ssl, "LOGIN_FAIL WrongPWHash\n"); return -1;
    }

    // 2. HOTP (세션 해시) 서버 측 계산 및 검증
    uint64_t V = u->R + u->counter; // 서버의 R, Counter 사용
    unsigned char Vbuf[8];
    for (int i = 0; i < 8; i++) Vbuf[7-i] = (V >> (i*8)) & 0xff;

    unsigned char calc_in[40];
    memcpy(calc_in, u->pw_hash, 32);
    memcpy(calc_in+32, Vbuf, 8);
    unsigned char sess_hash_srv[32];
    SHA256(calc_in, 40, sess_hash_srv);

    if (memcmp(sess_hash_srv, sess_hash_client, 32)!=0) {
        ssl_send(ssl, "LOGIN_FAIL WrongSession\n"); return -1; // 재전송 공격 등
감지
    }

    if (out_id) strcpy(out_id, id);
    return 1; // 인증 성공 (1 반환)
}

```

<Server> account.c

```

// handle_login 이 1(성공)을 반환했을 때 실행되는 부분
else if (auth == 1) {
    // 중복 로그인 방지
    if (lobby_is_logged_in(user_id)) {
        ssl_send(ssl, "ERR DuplicateLogin\n");
        goto cleanup;
    }
    // 3. 최종 승인 및 카운터 증가
    account_confirm_login(user_id); // 서버 DB 의 Counter 증가

```

```

    ssl_send(ssl, "LOGIN_OK\n");    // LOGIN_OK

    break; // 인증 루프 탈출 -> 로비 입장
}

```

3.4 퀴즈 진행 구현

3.4.1 로비 구현

퀴즈는 로비에서 시작되기에 로비에서의 메시지가 우선된다.

```

<Server> lobby.c
// 접속자 수 알림 메시지 전송
void broadcast_lobby_count() {
    int count = 0;
    for (int i = 0; i < MAX_CLIENTS; i++) if (clients[i].used) count++;

    char msg[64];
    sprintf(msg, "LOBBY_COUNT %d\n", count); // LOBBY_COUNT
    for (int i = 0; i < MAX_CLIENTS; i++)
        if (clients[i].used) ssl_send(clients[i].ssl, msg);
}

int lobby_add(SSL *ssl, const char *id) {
    // ... (유저 등록 로직) ...
    broadcast_lobby_count(); // 입장 시 인원수 갱신 전송

    // 첫 번째 유저(방장)인 경우
    if (admin_index == -1) {
        admin_index = i;
        ssl_send(ssl, "ADMIN_YOU_ARE_LEADER\n"); // ADMIN_YOU_ARE_LEADER

        char fl[1024];
        get_quiz_file_list(fl, sizeof(fl)); // 파일 목록 생성
        ssl_send(ssl, fl);                  // FILELIST
        ssl_send(ssl, "ADMIN_WAITING START_CMD\n");
    } else {
        // 일반 유저인 경우
        if (!game_started) ssl_send(ssl, "WAITING_FOR_START\n");
        // ...
    }
    // ...
}

```

Server는 LOBBY_COUNT, ADMIN_YOU_ARE_LEADER, FILELIST를 전송한다.

```

<Client>
// 로비 메시지 처리

```

```

if (!strcmp(buf, "LOBBY_COUNT", 11)) { // LOBBY_COUNT
    int count = 0; sscanf(buf, "LOBBY_COUNT %d", &count);
    printf("\r\033[2K[Lobby] 현재 접속자 수: %d 명\n", count);
    // ... 대기 ...
}
else if (!strcmp(buf, "ADMIN_YOU_ARE_LEADER", 20)) { // ADMIN_YOU_ARE_LEADER
    printf("\n[ADMIN] 당신은 방장입니다!\n");
}
else if (!strcmp(buf, "FILELIST", 8)) { // FILELIST
    // ... 파일 리스트 파싱 ...
}

```

Client는 LOBBY_COUNT, ADMIN_YOU_ARE_LEADER, FILELIST를 수신한다.

<Client>

```

void upload_quiz_file(SSL *ssl) {
    // ... 파일 열기 및 크기 측정 ...

    // 1. UPLOAD {name} {size}
    char cmd[512];
    sprintf(cmd, "UPLOAD %s %ld\n", filepath, filesize);
    SSL_write(ssl, cmd, strlen(cmd));

    // 2. UPLOAD_READY 수신
    char buf[256];
    int n = recv_line(ssl, buf, sizeof(buf));
    if (n > 0 && strcmp(buf, "UPLOAD_READY", 12) == 0) {
        printf(">> 전송 중...\n");
        // 3. 파일 바이너리 데이터 전송
        SSL_write(ssl, filebuf, filesize); // 파일 전송

        // 4. UPLOAD_OK 수신
        n = recv_line(ssl, buf, sizeof(buf));
        if (n > 0 && strcmp(buf, "UPLOAD_OK", 9) == 0) {
            printf(">> 업로드 완료! (서버에 암호화되어 저장됨)\n");
        }
    }
    // ...
}

```

Client는 UPLOAD 등으로 파일을 전송할 수 있다.

<Server> client_handler.c

```

void handle_upload(SSL *ssl, const char *line) {
    // ... 방장 권한 체크 ...

    // 1. UPLOAD 메시지 수신
    char filename[128]; int filesize;

```

```

sscanf(line, "UPLOAD %127s %d", filename, &filesize);

// 2. UPLOAD_READY
ssl_send(ssl, "UPLOAD_READY\n");

// 3. 파일 데이터 수신
char *filebuf = malloc(filesize);
int received = ssl_read_bytes(ssl, filebuf, filesize); // 정해진 크기만큼 읽기

// 4. UPLOAD_OK
if (encrypt_data_to_file(filename, filebuf, filesize) == 0) {
    ssl_send(ssl, "UPLOAD_OK\n"); // [송신] UPLOAD_OK
}
// ...
}

```

Server는 UPLOAD_READY, UPLOAD_OK 등으로 파일을 수신할 수 있다.

3.4.2 퀴즈 진행 구현

```

<Client>
// 방장이 START {filename} 메시지
if (input_state == 1) {
    // ... 파일 선택 ...
    char cmd[128]; sprintf(cmd, "START %s\n", fname);
    SSL_write(ssl, cmd, strlen(cmd));
}

// GAME_START 메시지 수신
if (!strncmp(buf, "GAME_START", 10)) {
    printf("\n!!! 게임이 시작되었습니다 !!!\n");
    // ENTER 메시지 전송
    SSL_write(ssl, "ENTER\n", 6);
    input_state = 0;
}

```

Client 중 방장이 START 메시지를 전송하면, Server의 GAME_START 메시지를 수신하고 ENTER 메시지를 전송한다.

```

<Server> lobby.c
void lobby_handle_start_command(SSL *ssl, const char *msg) {
    // ... 인원 수 체크 및 파일 로드 ...

    game_started = 1;
    // ... 점수 초기화 ...

    // 모든 클라이언트에게 시작 신호 전송
}

```

```

    lobby_broadcast("GAME_START\n"); // [송신] GAME_START
}

```

Server는 초기 작업을 진행한 후 GAME_START 메시지를 전송한다.

```

<Server> client_handler.c
// 각 클라이언트 스레드에서 ENTER 메시지 수신
if (!strcmp(line, "ENTER", 5)) {
    // 퀴즈 세션 함수로 진입 (여기서부터 퀴즈 프로토콜 시작)
    run_quiz_session(ssl, idx, lobby_get_quiz_file());

    // 퀴즈 종료 후 복귀하면 다시 로비 대기 상태
    // ...
}

```

Server의 클라이언트 스레드를 처리한 뒤, 본격적으로 퀴즈가 진행된다.

3.4.3 퀴즈 진행 구현

```

<Server> quiz.c
// run_quiz_session 함수 초입
char msg[256];
// INFORMSTART {Topic} {문제수} {이벤트여부} {난이도}
sprintf(msg, "INFORMSTART QuizTopic %d %d 1\n", quizCount, 0);
SendToClient(ssl, msg);

```

Server는 INFORMSTART를 전송한다.

```

<Server> quiz.c
int my_total_score = 0; // 퀴즈 루프 (for int i = start_index; i < quizCount; i++)
내부

char qmsg[4096];
// 난이도 표시 문자열 생성 ([상], [중]...)
const char *diff_str =
    (quizList[i].difficulty==3)?"[상]":(quizList[i].difficulty==2)?"[중]":"[하]";
char q_text_with_diff[1024];
sprintf(q_text_with_diff, "%s %s", diff_str, quizList[i].question);

// GIVEQ {번호}|{문제}|{보기 1}|{보기 2}|{보기 3}|{보기 4}|{이벤트}
sprintf(qmsg, "GIVEQ %d|%s|%s|%s|%s|%s|0\n",
    i+1, q_text_with_diff,
    quizList[i].options[0], quizList[i].options[1],
    quizList[i].options[2], quizList[i].options[3]);
SendToClient(ssl, qmsg);

```

Server는 GIVEQ로 위와 같은 형태로 문제를 전송한다.

<Client>

```
// run_quiz_async 함수 내 메시지 수신부
else if (!strncmp(buf, "GIVEQ", 5)) {
    // ... 파싱 변수 선언 및 strtok 파싱 ...
    // ...
    printf("\n-----\n");
    printf("[문제 %d] %s [제한시간: 15 초!]\n", q_num, q_text);
    if(evt) printf("★ EVENT 발동! 점수가 변동됩니다 ★\n");
    printf(" 1. %s\n 2. %s\n 3. %s\n 4. %s\n", op1, op2, op3, op4);
    printf("-----\n");

    // 입력 모드로 전환 및 카운트다운 초기화
    input_state = 2;
    remaining_time = 15;
    printf("[ ⌚ 남은 시간: %d 초] 정답(1~4): ", remaining_time);
    fflush(stdout);
}
}
```

Client는 GIVEQ를 이렇게 수신한다.

```
// run_quiz_async 함수 내 키보드 입력 처리부
else if (input_state == 2) {
    // ... 빈 입력 체크 ...
    int ans = atoi(line);
    if (ans < 1 || ans > 4) {
        // ... 유효성 검사 ...
        continue;
    }
    // SUBMITA {정답번호}
    char msg[64]; sprintf(msg, "SUBMITA %d\n", ans);
    SSL_write(ssl, msg, strlen(msg));
    // 메시지 전송 후 상태 변경 없이 대기 (결과 올 때까지)
}
}
```

Client는 SUBMITA를 전송한다.

```
// run_quiz_session 함수 내
char buf[128];
int selection = -1;

// [중요] 타임아웃(15 초) 적용된 읽기 함수 사용
int read_ret = ssl_readline_timeout(ssl, buf, sizeof(buf), QUIZ_TIMEOUT);

if (read_ret == -2) { // 타임아웃 발생 시
    selection = -1;
} else if (read_ret > 0) { // 메시지 수신 시
    trim(buf);
    if (strncmp(buf, "SUBMITA", 7) == 0) {
        int parsed_sel = -1;
        if (sscanf(buf, "SUBMITA %d", &parsed_sel) == 1) selection =
        parsed_sel;
    }
}
```

```

    }
} else { selection = -1; }

// 정답 확인 및 점수 계산 로직으로 이어짐
int correct = quizList[i].correct;
// ...

```

Server는 SUBMITA를 위와 같이 처리한다.

```

// run_quiz_session 함수 내 점수 계산 후
lobby_submit_answer(is_correct, quizList[i].difficulty, &score_gained,
&q_rank);
int overall_rank = lobby_save_score_and_get_total_rank(ssl, score_gained,
&my_total_score);

char gr[128];
// GRADE {정답여부} {정답} {획득점수} {문제순위} {누적총점} {전체순위}
sprintf(gr, "GRADE %d %d %d %d %d %d\n",
        is_correct, correct, score_gained, q_rank, my_total_score,
overall_rank);
SendToClient(ssl, gr);

// 모든 인원이 풀 때까지 대기 (Barrier)
lobby_wait_barrier();

```

Server는 GRADE 메시지를 전송한다.

```

// run_quiz_async 함수 내 메시지 수신부
else if (!strcmp(buf, "GRADE", 5)) {
    int is_corr, corr_ans, score, q_rank, total, overall_rank;
    sscanf(buf, "GRADE %d %d %d %d %d %d", &is_corr, &corr_ans, &score,
&q_rank, &total, &overall_rank);

    printf("\n"); // 카운트다운 줄바꿈

    if (input_state == 2) {
        // 아직 입력 중이었는데 결과가 옴 -> 시간 초과
        printf(">> 시간 초과... | 정답: %d 번\n", corr_ans);
    } else {
        // 정상 제출 후 결과
        if (is_corr) printf(">> 정답! (+%d 점) | 문제 순위: %d 위\n", score,
q_rank);
        else printf(">> 오답... (정답: %d 번)\n", corr_ans);
    }

    printf(">> [현재 총점: %d 점, 전체 순위: %d 위]\n", total, overall_rank);
    input_state = 0; // 대기 상태로 전환
}

```

Client는 GRADE 메시지를 위와 같이 처리한다.

```
// run_quiz_session 함수 마지막 (루프 종료 후)
pthread_mutex_lock(&highscore_mutex);
update_high_score(filename, my_total_score); // 최고 점수 갱신 시도
int record_high = get_high_score(filename); // 역대 최고 점수 조회
pthread_mutex_unlock(&highscore_mutex);

char endmsg[256];
// INFORMEND {최종점수} {역대최고기록}
sprintf(endmsg, "INFORMEND %d %d\n", my_total_score, record_high);
SendToClient(ssl, endmsg);

printf("[Quiz] Session finished for user %d.\n", user_id);
```

Server는 모든 퀴즈가 끝나면 quiz.c를 전송한다.

```
// run_quiz_async 함수 내
else if (!strcmp(buf, "INFORMEND", 9)) {
    int final_score, record;
    sscanf(buf, "INFORMEND %d %d", &final_score, &record);

    printf("\n=== 퀴즈 종료 ===\n");
    printf("나의 최종 점수: %d 점\n", final_score);
    printf("이 퀴즈의 역대 최고 기록: %d 점\n", record);
    printf(">> 로비에서 대기합니다...\n");

    input_state = 0; // 로비 대기 모드
}
```

Client는 퀴즈가 끝나면 로비로 돌아간다.

4. 결론

'타조'는 보안네트워크프로그래밍 수업에서 배운 내용을 바탕으로 학습용 보조 퀴즈 프로그램 'Let's Quiz'를 구현하였다. 이를 통해 학생들이 다같이, 즐겁게 공부할 수 있기를 기대한다.